Daffodil International University

LAB REPORT

Course Title: Computer Graphics Lab
Course Code: CSE412 & CSE422

TOPICS: Implementation of DDA and Bresenham's Line Drawing Algorithms.

SUBMITTED TO:

Course Teacher : Mr. Abdullah Al Mamun
Designation : Lecturer (Senior Scale)

Department of Computer Science and Engineering
Daffodil International University

SUBMITTED BY

NAME: MD. Mahmudur Rahman Supto
STUDENT ID: 0242310005101419
SECTION: 64_E1
DEPARTMENT: CSE,
Daffodil International University

1 Objective

- To understand line drawing algorithms.
- To implement DDA and Bresenham algorithms.
- To compare efficiency and accuracy of both algorithms.

2 Tools

- Programming Language: C++
- Graphics Library: OpenGL (GLUT)
- IDE: CodeBlocks / GCC

3 Theory

DDA Algorithm: The Digital Differential Analyzer (DDA) algorithm uses incremental floating-point calculations to draw a line. It calculates intermediate points using slope.

Bresenham Algorithm: Bresenham's algorithm is an efficient integer-based algorithm that avoids floating-point operations and uses decision parameters.

4 Example Used Functions

- `glBegin(GL_POINTS)`
- `glVertex2i()`
- `glEnd()`
- `glColor3f()`
- `glFlush()`

5 Program Code (DDA)

```
#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

int x1_, y1_, x2_, y2_;

// Plot pixel
void drawPixel(int x, int y) {
    glBegin(GL_POINTS);
        glVertex2i(x, y);
    glEnd();
```

```
}

// DDA Algorithm
void drawLineDDA() {
    int dx = x2_ - x1_;
    int dy = y2_ - y1_;

    int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);

    float xInc = dx / (float)steps;
    float yInc = dy / (float)steps;

    float x = x1_;
    float y = y1_;

    cout << "\nVisited Points (DDA):\n";

    for (int i = 0; i <= steps; i++) {
        int px = round(x);
        int py = round(y);

        cout << "(" << px << ", " << py << ")\n";

        drawPixel(px, py);

        x += xInc;
        y += yInc;
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(3);

    glColor3f(1, 1, 1);
    drawLineDDA();

    glFlush();
}

void init() {
    glClearColor(0, 0, 0, 1);
    gluOrtho2D(-300, 300, -300, 300);
}

int main(int argc, char** argv) {
    cout << "Enter x1 y1: ";
    cin >> x1_ >> y1_;
    cout << "Enter x2 y2: ";
    cin >> x2_ >> y2_;
```

```
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("DDA Line");

    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}
```

6 Output (DDA)

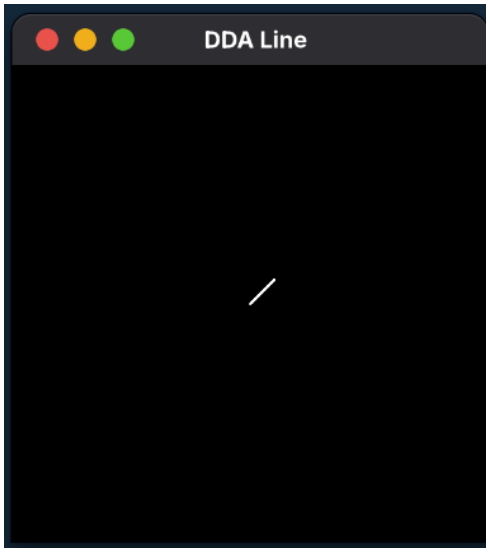

Figure 1: DDA Output - Case 1

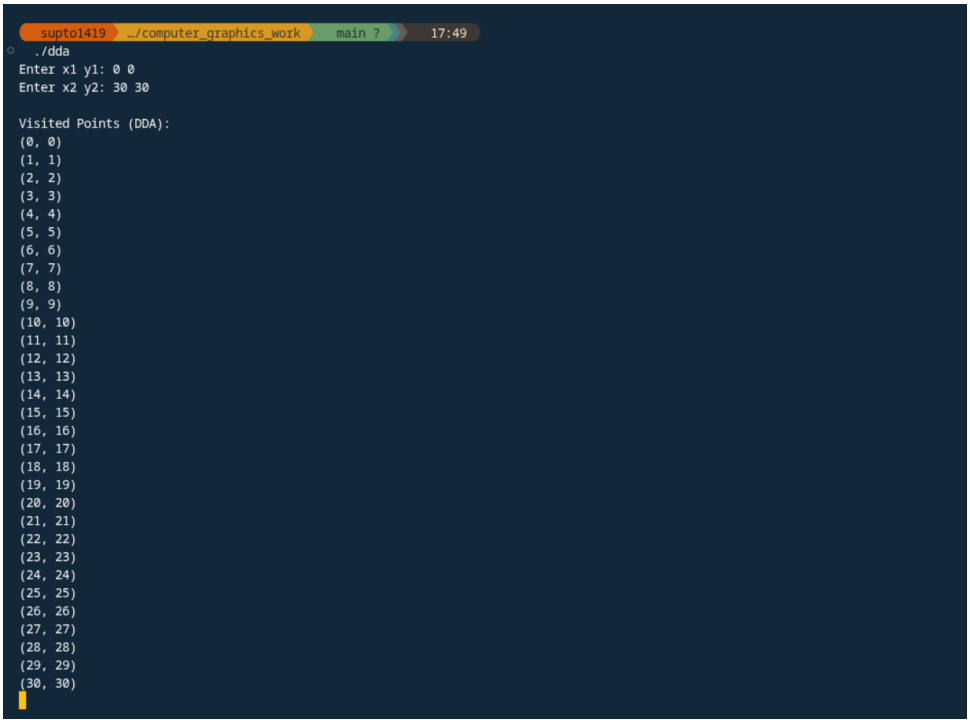

Figure 2: DDA Output - Case 2

7 Program Code (Bresenham)

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

int x1_, y1_, x2_, y2_;

// Plot pixel
void drawPixel(int x, int y) {
    glBegin(GL_POINTS);
        glVertex2i(x, y);
    glEnd();
}

// Bresenham Algorithm
void drawLineBresenham() {
    int dx = abs(x2_ - x1_);
    int dy = abs(y2_ - y1_);

    int sx = (x1_ < x2_) ? 1 : -1;
    int sy = (y1_ < y2_) ? 1 : -1;

    int err = dx - dy;

    int x = x1_;
    int y = y1_;
```

```
    cout << "\nVisited Points (Bresenham):\n";

    while (true) {
        cout << "(" << x << ", " << y << ")\n";
        drawPixel(x, y);

        if (x == x2_ && y == y2_) break;

        int e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x += sx;
        }

        if (e2 < dx) {
            err += dx;
            y += sy;
        }
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(3);

    glColor3f(1, 1, 1);
    drawLineBresenham();

    glFlush();
}

void init() {
    glClearColor(0, 0, 0, 1);
    gluOrtho2D(-300, 300, -300, 300);
}

int main(int argc, char** argv) {
    cout << "Enter x1 y1: ";
    cin >> x1_ >> y1_;
    cout << "Enter x2 y2: ";
    cin >> x2_ >> y2_;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Bresenham Line");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
```

```
    return 0;
}
```

8 Output (Bresenham)

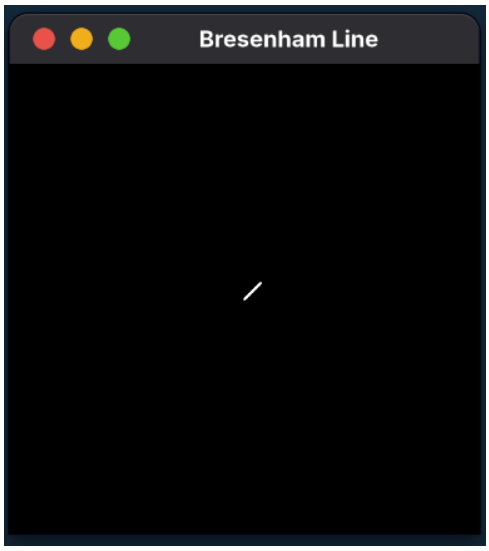

Figure 3: Bresenham Output - Case 1

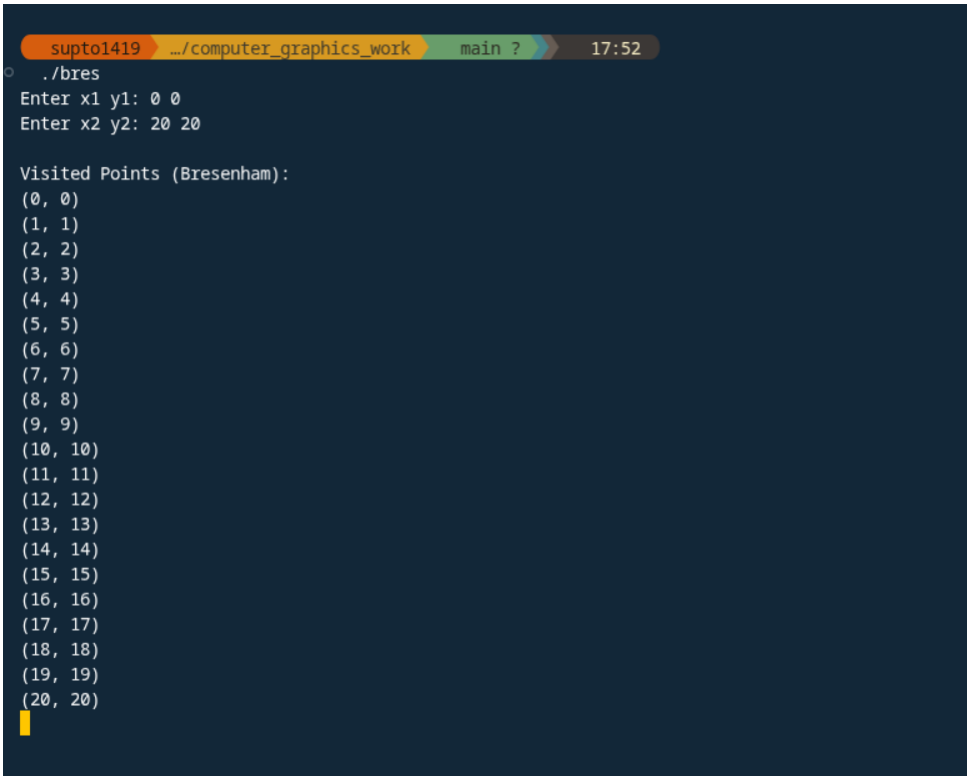

Figure 4: Bresenham Output - Case 2

9 Result

The experiment was successfully completed by implementing both the DDA and Bresenham line drawing algorithms. The generated outputs confirmed that both algorithms are capable of producing straight lines between two given points.

During observation, it was found that the DDA algorithm uses floating-point arithmetic to calculate intermediate points, which may introduce rounding errors in some cases. However, it is simple and easy to understand.

On the other hand, Bresenham's algorithm uses only integer calculations, making it more efficient and faster. It produced more accurate and visually consistent lines without floating-point inaccuracies.

From multiple test cases, it was observed that Bresenham's algorithm performs better in terms of speed and precision, especially when drawing long lines or working in real-time graphics systems.

10 Conclusion

This experiment provided a clear understanding of fundamental line drawing algorithms used in computer graphics. By implementing both DDA and Bresenham algorithms, we gained practical knowledge of how lines are rasterized on a pixel-based display.

The experiment demonstrated that although the DDA algorithm is easier to implement, it relies on floating-point operations, which can reduce efficiency and accuracy. In contrast, Bresenham's algorithm is more optimized as it uses only integer arithmetic, making it faster and more suitable for real-time applications.

Overall, this lab enhanced our understanding of graphics rendering techniques, coordinate systems, and algorithm optimization. It also highlighted the importance of choosing efficient algorithms in computer graphics applications.